

When Does an Assignment Solver Beat a Sort Key? Dispatching Under Queue-Time Constraints in a Simulated 300 mm Wafer Fab

Alton Alexander · Front Analytics · alton@frontanalytics.com

Draft for review, September 2026 · Code and data: <https://github.com/altonalexander/fab-optimization> (release v0.2.0; this document built at commit `ecf1b9f`)

Abstract. Every time a machine in a wafer fab frees up, a dispatching decision picks the next lot. Almost all production fabs make that decision with a sort key — a priority rule that ranks the waiting lots one at a time. The alternative is to solve an assignment across all waiting lots and all free tools at once. This paper asks a narrow question: on a realistic simulated fab, does the assignment formulation beat the best sort key we could build, and by how much? We use the public SMT2020 low-volume/high-mix testbed inside the PySCFabSim discrete-event simulator, add enforcement of the queue-time windows the dataset already carries (with rework on violation and scrap after repeated failure), and compare FIFO, critical ratio, a queue-time-aware sort key we call QT, and a per-family CP-SAT assignment solver, all resumed from one shared warmed checkpoint over 180 simulated days at full load. Two results. First, the choice of sort key decides whether the fab is viable at all, not merely how efficient it is: QT holds work-in-process stationary with zero scrap and 82-94 % on-time across three seeds, while FIFO and critical ratio diverge on the same fab at a consistent +10 lots/day. Second, the solver first lost decisively — 48 good lots/day against 57.5 and 25 % on-time against 82 % — and the cause was a single constant: its queue-time term was written for windows measured in minutes and applied to windows of 10-240 hours, so a typical at-risk lot received a $1.01\times$ preference against a due-date term reaching $50\times$. Made window-relative, the solver matches QT on throughput, violations, scrap and stability and beats the tuned QT by 6.5 on-time points with $6.8\times$ less total tardiness, at roughly $5\times$ the wall-clock cost. A claim that the solver won by rebalancing across products did not survive tuning the baseline and is withdrawn. We report the measurement failures alongside the results, because each was caught by a conservation check or a control rather than by a better metric, and because one of them — an unchecked constant inverting a replicated conclusion — bears directly on how “the solver lost” should be read anywhere.

1. Introduction

A 300 mm wafer fab runs a few hundred product routes of several hundred steps each through roughly a thousand tools grouped into families of interchangeable machines. Routes are re-entrant: the same lot returns to the same lithography, etch and diffusion families dozens of times, so the queue a lot joins depends on every dispatching decision made before it. Tools break down, need preventive maintenance, carry recipe-dependent setups, and in the furnace families process batches that must be filled. Whenever a tool frees, something has to decide which of the waiting lots it takes. That decision — the dispatching rule — is made tens of thousands of times a day.

Production fabs make it with a sort key. Each lot in the family’s queue is scored by a tuple (a hot-lot flag, a setup match, an age, a ratio of time-to-due-date over remaining work) and the tool takes the top of the sort. FIFO and critical ratio (CR) are the canonical instances; the survey literature on job-shop dispatching rules is large and old [1,2], and the semiconductor-specific reviews [3,4] describe the same family of policies under the constraints particular to wafer fabrication.

The alternative treats the decision as an assignment: at each moment, choose which lot runs on which free tool across a whole family (or the whole fab) so as to minimise a cost, subject to capacity, batch and resource constraints. Constraint programming solvers such as CP-SAT [5] make such models tractable at the sub-second timescale a dispatcher needs. The intuition for why assignment should win is that a sort key can only rank: it cannot express “these two lots should share a furnace batch,” “this lot should wait because the tool it is on is the only one that can run the lot behind it,” or “give this product a little less service because that one is about to miss.” An assignment over a set can.

The question this paper asks is deliberately narrow. Not “is scheduling useful” — a sort key is already a scheduler — but: **on a realistic fab, does solving an assignment at the dispatch moment beat ranking, and by how much, against the best sort key we can construct?** The question is hard to answer in a real fab because the demand, the breakdowns and the tool set are never the same twice, so a policy that looks better this quarter may have had an easier quarter. A simulator on a public testbed removes that confound: every policy runs on identical demand, identical breakdowns and an identical starting fab, so the only thing differing between two rows of a results table is the decision.

We make five contributions.

1. **Queue-time enforcement on SMT2020.** The dataset ships queue-time windows on 264 route steps; the simulator parses and ignores them. We enforce them — a lot that misses its window is reworked from the step that opened it, and a lot that misses too often is scrapped — which gives the fab a way to lose work rather than merely be slow (§3.3).
2. **The finding that the sort key decides viability.** With queue times enforced, a queue-time-aware sort key holds the fab stationary at full load with zero scrap; FIFO and CR diverge on the same fab. Replicated on three seeds (§6.1).
3. **A minimum viable assignment solver.** Once its queue-time term is calibrated to the fab’s window lengths, the CP-SAT solver matches the best sort key on throughput, violations, scrap and stability and beats it on lateness (§6.4–6.6).
4. **A record of how the measurement went wrong,** including the reversal from “the solver loses decisively” to “the solver wins”: a constant off by two orders of magnitude relative to the data (§6.3), a short-window transient that would have reported the opposite answer (§7.3), and a baseline-containment effect that makes any solver-versus-rule comparison silently unfair when the rule is tuned (§7.4).
5. **A reproducible testbed.** Every number here comes from result files in the repository, every table in this document is generated from them, and the sequence of decisions is recorded in architecture decision records that preserve the falsified verdicts rather than tidying them [6].

2. The testbed

2.1 Dataset

SMT2020 [7] is a public semiconductor manufacturing testbed with four scenarios. We use the low-volume/high-mix (LVHM) scenario throughout. Read into the simulator it presents:

- **10 products**, with routes of 242 to 583 steps;
- **1,313 tools**, of which **913** are process tools in **105 families** (median family size 5, the largest 118) and **400** are `Delay` pseudo-tools that model fixed waits rather than capacity;
- an initial work-in-process of **2,164 lots**, and a release schedule (`order.txt`) that the simulator follows verbatim at **≈ 56.6 lots/day**;
- **264 distinct steps carrying a queue-time window**, with nine distinct window lengths between **1 and 24 hours**;
- batch (furnace) steps on 9–19 steps per route, and setup-bearing steps on 27–58 per route.

product	route steps	steps with a q-time window	batch steps	setup-bearing steps
product 1	521	29	15	48
product 2	529	38	19	54
product 3	583	41	17	58
product 4	343	25	11	35
product 5	242	11	9	27
product 6	293	27	12	30
product 7	353	24	12	36
product 8	375	20	12	40
product 9	384	23	13	34
product 10	390	26	15	39

Table 1. Route structure of the ten LVHM products as loaded by the simulator. Queue-time steps are the ones on which a window opens.

2.2 Simulator

PySCFabSim [8] is an open discrete-event simulator built for this testbed. It models re-entrant routes with the route’s own rework, setup matrices with minimum-run-length rules, batching (we use its `Demand` batch strategy), time- and piece-based preventive maintenance, random breakdowns and repair, and due dates per lot. We vendor it unchanged at a pinned upstream commit and make our additions in a small number of clearly marked places; a document in the repository lists every divergence.

The simulator simplifies in ways that matter for the reading of any result [9]:

- **Transport time is zero** in this configuration. Nothing moves between tools; a lot is available at its next family the instant it finishes. (This is why a reticle-exclusivity constraint we tested earlier never bound — a mask move is free.)
- **Delay steps are a pseudo-toolset** of 400 stations rather than a modelled wait.

- **There is no storage or stocker capacity.**
- **Queue-time constraints are parsed and ignored** upstream. This is the gap §3.3 closes.

2.3 What is and is not in play

Two constraint overlays built earlier in this project — tool qualification (dedication) and reticle exclusivity — are **off** in every experiment here. Both were tested and neither separated the rules: qualification only filters which tools a lot may use, which a sort key handles; reticles never bind on this fab because there are enough masks and moving one costs nothing. The tool set is the dataset's full complement; an earlier attempt to trim it showed the fab has about 5 % of slack, so capacity is a cliff rather than a dial.

What is in play, in every run: re-entrant routes, the route's own rework, setups with minimum runs, batching, PM, breakdowns, due dates, family qualification (a step runs only on its own family), and — added here — queue-time windows with rework and scrap.

3. Method

3.1 Warm-up, checkpoints and the A/B

A discrete-event simulator cannot start at day 90; it must simulate there. We warm the fab for 90 simulated days under one rule, checkpoint the entire simulator state — event queue, tool setups, pending breakdowns, RNG — and resume **every** policy from that identical checkpoint. The rows of a results table therefore differ in nothing but the dispatching decision, which is the only reason the table means anything. The checkpoint is keyed by every setting that changes the trajectory (dataset, seed, warm-up rule, batch strategy, day, and every constraint in force), so a fab warmed under one constraint can never be silently resumed under another.

Measurement runs for a further **180 days** (days 90-270). KPIs are computed over lots that completed in the window: throughput, cycle time, on-time delivery, total tardiness in lot-days, tool utilisation, and — from the additions below — violations, reworks and scrapped lots. All are reported per product as well as fab-wide, because a fab-wide average hid three effects during this work before we stopped trusting it.

3.2 Admissibility before interpretation

A diverging fab produces numbers that describe where the window was cut, not the policy. Every cell is therefore gated before it is read:

1. **Conservation.** Over the window, $\text{releases} = \text{good lots out} + \text{scrapped} + \Delta\text{WIP}$. The release schedule is fixed, so this must reproduce ≈ 56.6 lots/day (57.2 in every warmed run reported here). This identity caught a reporting error in which queue-time counters were accumulated from day 0 while throughput was window-scoped, implying 8,325 lots left a fab that had started 5,100.
2. **Stationarity.** The WIP slope over the final third of the window, in lots/day. A cell that is still climbing is reported as diverging and its other numbers are not interpreted.

3. **Not stationary by loss.** Scrap destroys material, so a fab can hold WIP flat because it is eating itself. A cell losing more than a fifth of its releases is not a fab anyone would run, however steady its WIP.

3.3 Queue-time enforcement, rework and scrap

A queue-time window is a limit on how long a lot may wait between two specific steps — typically after a clean or a bake — before the work is damaged. SMT2020 carries these limits on 264 steps; PySCFabSim reads them into `Step.cqt_for_step` and `Step.cqt_time` and does nothing with them.

Our enforcement reads the clock at the industry definition: the window opens when the opening step starts processing, and is checked when the closing step starts processing. A window length is `cqt_time × s`, where `s` is a scale we sweep; the results here use `s = 10`, so windows are 10–240 hours. On violation the lot is **reworked**: its processed steps are rolled back to the opening step and it redoes that work. The route’s own rework is modelled identically and reuses the mechanism.

The first version reworked without limit, and it turned the fab into an absorbing state: over 30 cold days at `s = 8`, 422 violations fell on twelve lots, six of which were reworked twenty or more times and one 83 times, concentrated on six step orders. Those lots never left; WIP climbed while fab-wide utilisation fell to 30 %, because trapped work on a handful of steps starved the rest. That is not a fab, it is a missing termination rule. A real fab scraps material that has failed too often, and the scrap is precisely the loss this constraint class is supposed to introduce. We therefore **scrap** a lot after its third rework (`cqt_max_rework = 3`): it leaves the active set and is deliberately not counted as a completion, so it reduces throughput and never inflates on-time.

Three controls establish that the machinery has no side effects of its own:

arm (40 cold days, seed 0)	lots out	util %	violations	reworks	scrapped	end WIP
no enforcement	2380	81.9	0	0	None	2,054
scale 8, detection only	2380	81.9	300	0	None	2,054
scale 8, rework, uncapped	1932	81.6	734	733	None	2,502
scale 4, rework, uncapped	700	44.4	14518	14509	None	3,734
scale 4, rework, cap 3	1612	61.8	5216	4453	752	2,070

Table 2. Mechanism checks over 40 cold days from an empty fab, seed 0. Detection-only is digit-identical to no enforcement on every field. Uncapped rework at scale 4 collapses utilisation; the cap recovers it and converts the trapped work into scrap.

3.4 The nondeterminism of solver runs

CP-SAT is configured with a fixed random seed and, per the code’s own comment, for “bit-identical output for identical input.” It is not bit-identical, because its per-solve budget is a **wall-clock** limit (5 ms per family), so how much search completes depends on machine load. Four identical serial solver runs on a 3-day probe returned 169, 169, 176 and 181 lots and on-time of 84.62, 84.62, 84.66 and 85.64 %. Two consequences follow. A single solver row against a single rule row cannot be read, so solver results below are replicated. And equivalence of the parallel planner (§4.3) had to be shown statistically — parallel runs land inside the serial spread — rather than by fingerprint.

The deterministic-time budget OR-Tools offers would fix this and is left as future work because changing it mid-experiment would shift the effective budget.

4. Dispatching policies

Every policy is consulted at the same point: a tool has freed, and the simulator asks for a priority tuple per waiting lot in that tool's family, then takes the minimum. The upstream tuples all begin with a minimum-run-length flag and the setup time a lot would incur on this tool, so setup avoidance is common to every rule.

4.1 FIFO and critical ratio

`fifo` orders by hot-lot priority, then release time. `cr` orders by priority, then the critical ratio $(\text{due} - \text{now}) / \text{remaining processing}$. Neither is aware that queue-time windows exist, and — since the windows are hours while due dates are days — neither will prioritise a lot two hours from lapsing over one that is merely old.

4.2 QT: a queue-time-aware sort key

`qt` is `cr` with one tier inserted ahead of setup: a lot with an **open, still-achievable** queue-time window outranks every lot without one, ordered among themselves by slack. Everything else is exactly `cr`, so `qt - cr` isolates the value of protecting windows and nothing else. Upstream had written this tier into all five of its rules and left it commented out (`#0 if lot.cqt_waiting is not None else 1`) — reasonably, since it had nothing to act on — but theirs is binary and ours orders by slack.

Two details of the rule are results in their own right.

Lapsed lots must not be promoted. The first version ordered by slack alone, which puts the most hopelessly late lots at the front of every queue. On a warmed fab that is most of them: of 3,095 active lots, 978 held an open window and 650 of those (66.5 %) were already past their deadline, with median slack -67 h and the worst -262 h. A window that has lapsed cannot be un-lapsed, so promoting such a lot spends capacity on work already guaranteed to rework or scrap. The bug was invisible on a cold fab (few lots lapse in twelve days; `qt` cost nothing there — 617 lots in both arms, violations $42 \rightarrow 9$) and catastrophic on a warmed one (throughput down 55 %). Fixed, only lots with positive slack are promoted; a lapsed lot takes its turn by `cr`. We measured, and rejected, two other explanations first: setup thrash (setup is 0.83 % vs 0.90 % of tool time in the two arms — far too small to explain the loss) and batch-breaking (throughput was identical cold).

Promotion should be selective. Promoting any saveable at-risk lot, whether it has twenty minutes or two hundred hours left, is indiscriminate and disrupts setup and batch grouping for lots that were never in danger. A threshold — promote only when $\text{slack} < f \times \text{window}$ — is the tuned version. Both $f = 1.0$ (promote all) and $f = 0.50, 0.25$ are reported; the default remains 1.0 so that earlier rows are unchanged.

4.3 SLATE: a per-family CP-SAT assignment

The solver is the project's original object. Its model is a single-period assignment: one Boolean per feasible (lot, tool) pair, at-most-one tool per lot, tool capacity, batch-furnace firing bounds, and reticle exclusivity. There is **no time index**: it assigns, it does not

sequence, on the argument that re-solving every 60 simulated seconds makes the present the only moment that matters. The objective minimises

```
cost(lot, tool) = (setup_s + process_s) / (urgency × qtime_boost)
```

where `urgency` collapses everything Python knows into one scalar — a piecewise critical-ratio curve (gentle above $CR = 1$, steep below it, capped at $50\times$), an ageing term, and a downstream-congestion term that discounts pulling a lot into an already-congested next family — and `qtime_boost = 1 + 600 / max(slack_s, 60)`. An unassigned lot is penalised in proportion to a related urgency that uses `1 + 3600 / max(slack_s, 60)`. That the two constants differ, where the design says the second should be the linearisation of the first, is noted and left as found.

Three engineering properties matter for reading the results:

- **Decomposition.** The eligibility matrix is block-diagonal — a step runs only on its family — so the fab-wide problem is solved as ~ 105 independent per-family problems of ~ 25 lots $\times \sim 12$ tools each, with lazy invalidation so only families whose state moved are re-solved. This is what makes $\sim 240,000$ rebuilds per 180 days tractable at all.
- **Coverage.** The solver plans on a 60-second cycle; a lot the current slate has no token for is dispatched by a **fallback** rule. Coverage — the share of decisions the solver actually made — is $\approx 46\%$ in every solver run here. **More than half of a solver run is its fallback.** The fallback is selectable (`score`, `cr`, `qt`), and the choice turns out to matter enormously (§7.4).
- **Parallelism.** The per-family solves were serial; we made them concurrent (one backend instance per worker, results merged in a fixed order), for a measured **$2.9\times$** on a 3-day probe. Accepted on statistical equivalence per §3.4.

5. Experimental design

Operating point. $1.00\times$ the dataset’s release schedule; queue-time scale 10; rework on; scrap after three failures; the full tool set; no qualification or reticle overlay. The warm-up rule is `qt` (so the starting fab is one a viable rule would actually produce), and every row resumes the same checkpoint. Seeds 0, 1 and 2 for the rules; seed 0 for the solver.

Pre-registered bar. Before the solver was run against `qt` we fixed what “winning” would mean: (i) keep scrap at zero and WIP stationary; (ii) hold good lots/day within run-to-run noise of `qt`’s; (iii) close some of the per-product on-time spread. Landing on `qt`’s numbers would mean a sort key with extra steps at several times the compute.

Replication. Rules: three seeds. Solver: two replicates at seed 0 for each configuration except the final symmetric one, for which one run has been completed at the time of writing.

Progress instrumentation. A 180-day solver run takes hours and, until we added it, printed nothing until it wrote its result. Every run here reports WIP, throughput rate, scrap and solver coverage every ten simulated days, which is what allowed a failing configuration to be recognised at minute ten rather than hour four.

6. Results

6.1 The sort key decides whether the fab is viable

seed	rule	good/ day	on- time	CT (d)	tardiness	scrap/ day	viol/ day	WIP	slope, final $\frac{1}{3}$
0	FIFO	40.2	22.53%	59.4	154,688	6.3	60.0	2,199→4,117	-0.95
0	CR	44.4	15.16%	48.9	82,296	3.8	88.6	2,199→3,822	+10.38
0	QT	57.5	81.66%	38.4	1,925	0.0	1.4	2,199→2,145	-1.13
1	FIFO	41.8	24.65%	56.1	136,573	0.8	32.9	2,130→4,766	+15.40
1	CR	46.6	34.01%	45.1	56,288	2.6	74.1	2,130→3,567	+10.51
1	QT	56.8	92.70%	38.0	590	0.0	1.1	2,130→2,207	+2.08
2	FIFO	41.5	22.44%	58.1	150,370	0.7	33.4	2,239→4,934	+15.93
2	CR	46.1	22.85%	47.1	72,154	3.0	72.8	2,239→3,695	+10.17
2	QT	57.6	93.95%	37.8	353	0.0	1.3	2,239→2,164	+0.57

Table 3. Three rules on three seeds, each triple resumed from its own qt-warmed checkpoint at full load. Slope is WIP lots/day over the final third of the 180-day window.

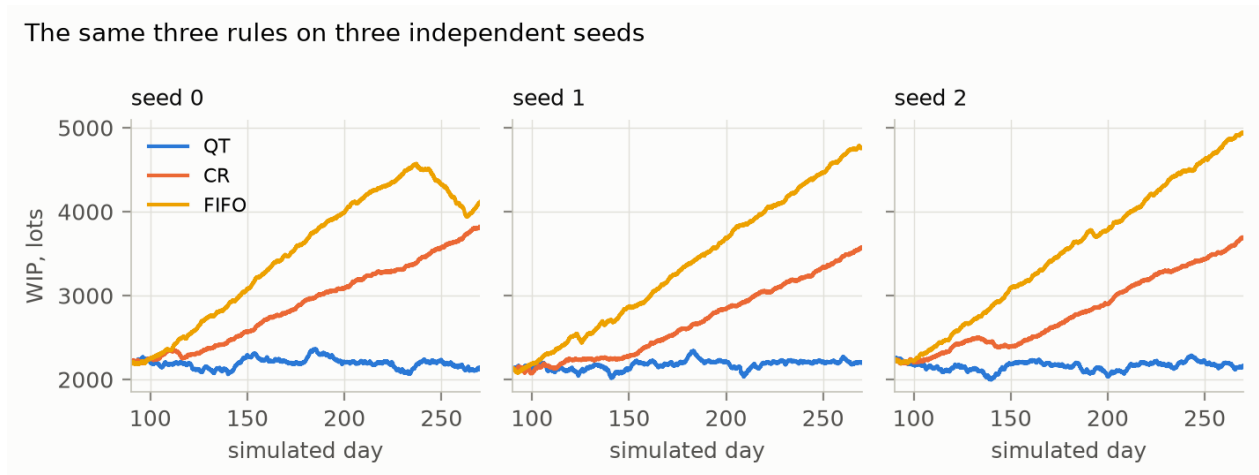


Figure 1. Work-in-process over the measurement window for the three sort keys on three independent seeds. QT holds WIP at its starting level on every seed; CR climbs at a strikingly consistent +10 lots/day; FIFO climbs faster and, on seed 0, saturates at a high level by scrapping.

With queue-time windows enforced, `qt` holds WIP stationary on all three seeds (final-third slopes -1.13 , $+2.08$, $+0.57$ lots/day), scraps nothing, holds violations near one a day, and delivers 82–94 % on-time. On the same fab, same demand and same machines, `cr` diverges at $+10.2$ to $+10.5$ lots/day on every seed, and `fifo` diverges on two seeds and on the third saturates at a high WIP by scrapping 6.3 lots/day. Total tardiness differs by factors of 40–80.

This is the larger result of the paper and it was not the one we set out to find. The dispatching rule is usually discussed as a matter of efficiency — a few percent of cycle time or on-time. Here it is a matter of **stability**: the mechanism is that a missed window reworks the lot, the rework competes with fresh work, more windows are missed, and the backlog compounds. `cr` and `fifo` never break the loop because neither can see a window; `qt` breaks it by spending capacity only where it still buys something. We had

initially read the divergence as a capacity limit — “rework is demand this fab has no slack to absorb” — and that reading was wrong: `cr` diverges at +10 lots/day on the same tools that `qt` holds flat.

6.2 The solver, as first measured, loses decisively

Against the standing `qt` row, two replicates of the solver with its queue-time term as shipped:

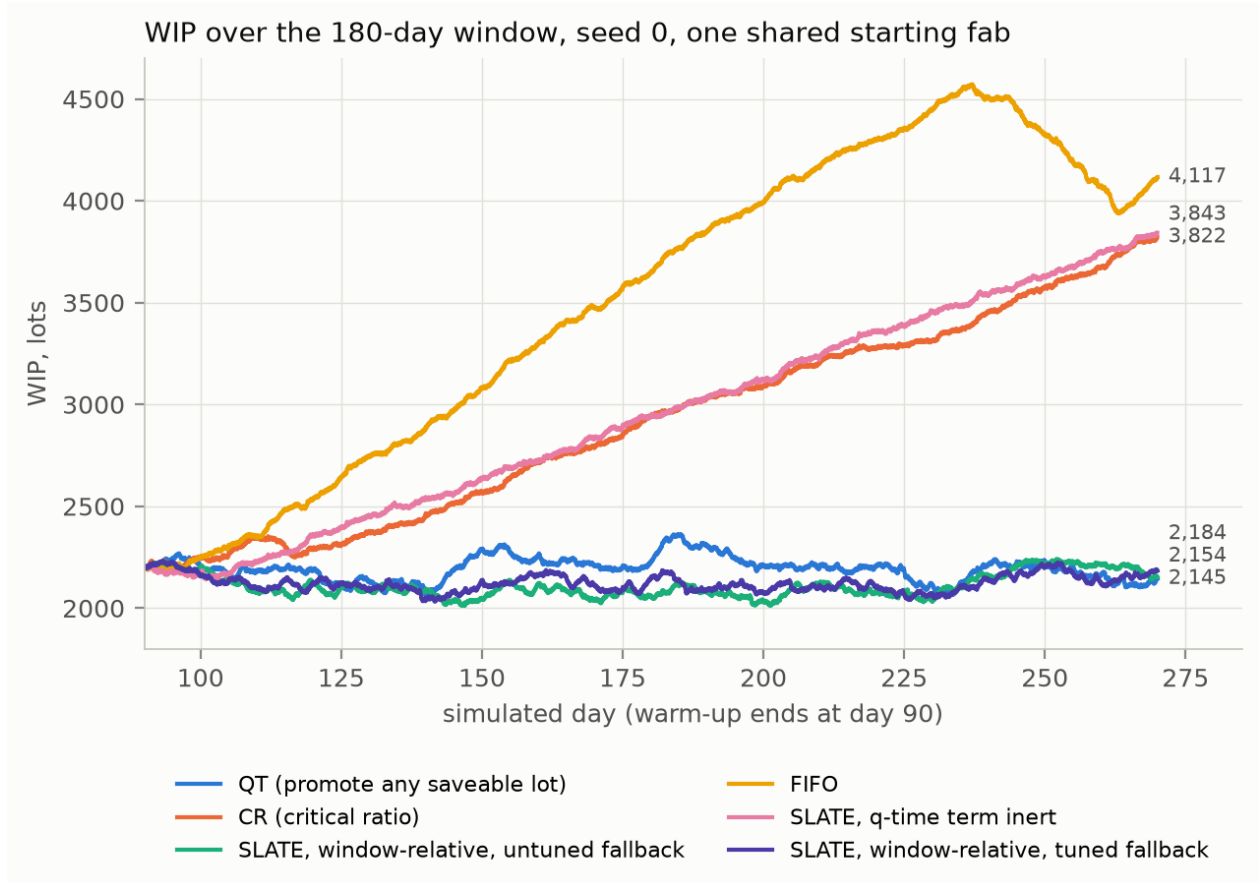


Figure 2. WIP over the window for every configuration on seed 0, all resumed from the same checkpoint. The solver with its shipped queue-time term (pink) tracks CR (orange) almost exactly; the solver with the window-relative term (green, violet) tracks QT (blue).

The solver delivered 48.1 and 48.5 good lots/day against `qt`’s 57.5, 24.9 and 24.2 % on-time against 81.7 %, tardiness of ~91,700 lot-days against 1,925, with WIP diverging at +10.4 and +9.0 lots/day. The two replicates agree to within 0.4 lots/day, so this was not noise. All three pre-registered bars failed, and the per-product spread did not close but collapsed: nine of ten products fell by 53 to 81 points. We wrote this up, with numbers and replicates, as the fourth constraint class on which the assignment formulation had lost.

The tell we should have read sooner is in Figure 2: the solver’s divergence signature is nearly identical to `cr`’s — +10.40 vs +10.38 lots/day, ending at WIP 3,843 vs 3,822. With a queue-time term supposedly active and a `qt` fallback on half its decisions, the solver was behaving exactly like the rule that cannot see windows.

6.3 The constant

An optimiser should never lose to a heuristic it could reproduce; if it does, the objective is not encoding what is wanted. We therefore asked what the objective actually told the solver about queue time.

slack remaining	<code>cost()</code> , 600/ slack	CP-SAT, 3600/slack	window-relative, 24 h window	window-relative, 240 h window
1 minute	11.000×	61.000×	61.00×	61.00×
10 minutes	2.000×	7.000×	61.00×	61.00×
1 hour	1.167×	2.000×	61.00×	61.00×
4 hours	1.042×	1.250×	37.00×	61.00×
16 hours (p75 of saveable at-risk lots)	1.010×	1.062×	9.89×	61.00×
2 days	1.003×	1.021×	—	31.00×
10 days	1.001×	1.004×	—	7.00×

Table 4. The priority boost a lot receives from the two shipped queue-time terms as a function of remaining slack, and the CP-SAT term after the slack is expressed as a fraction of the lot's window ($1 + 6/\text{frac}$, saturating at $61\times$ once less than a tenth of the window remains). The shipped terms are written for windows measured in minutes; a lot with 16 hours of slack in a 24-hour window moves from $1.06\times$ to $9.9\times$.

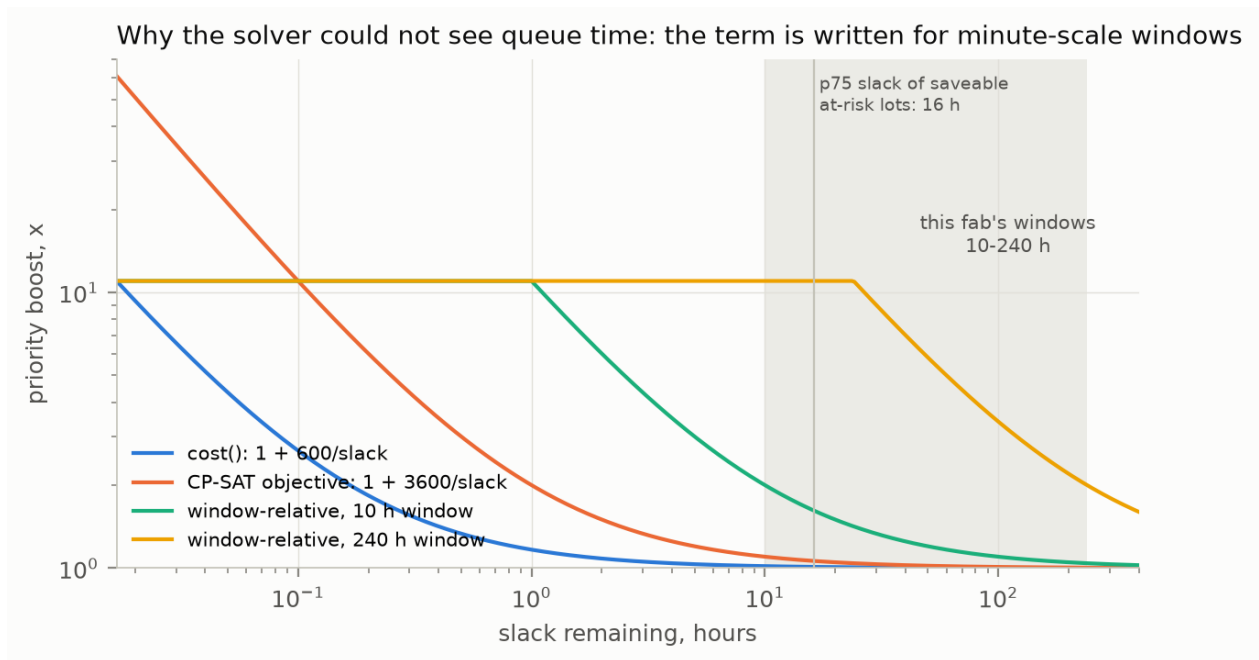


Figure 3. The queue-time boost against remaining slack on log axes. The shipped terms (blue, orange) decay to nothing by a few hours. This fab's windows are 10–240 hours (shaded); the typical saveable at-risk lot has 16 hours of slack and receives $1.01\times$ and $1.06\times$. The window-relative form (green, yellow) treats a lot near the end of a 10-hour window and one near the end of a 240-hour window alike.

$1 + 600/\text{slack}$ is $11\times$ at one minute of slack, $2\times$ at ten minutes and $1.17\times$ at one hour. This fab's windows are 10–240 hours, and the measured p75 of saveable at-risk lots is 16 hours of slack, priced at **1.010×** by `cost()` and **1.063×** by the CP-SAT objective — against a due-date urgency in the same product that reaches $50\times$. The term was not

weak; it was arithmetically absent. Every conclusion in §6.2 was drawn from a solver that had, in effect, never been told about queue time.

The fix is one line of data and no change to the solver: pass $600 \times \text{slack} / \text{window}$ instead of raw slack, so that both terms become window-relative ($\text{cost}() \rightarrow 1 + 1/\text{frac}$; the CP-SAT objective $\rightarrow 1 + 6/\text{frac}$). A lapsed window still reports the inert sentinel, for the reason §4.2 gives — the $\max(\text{slack}, 60)$ clamp would otherwise turn a lot 200 hours past deadline into the maximum boost and send the solver chasing the most hopeless work in the fab, which is exactly the qt bug.

6.4 The solver, corrected

configuration	good/ day	on- time	CT (d)	tardiness	scrap/ day	viol/ day	util %	end WIP	slope	wall (s)
FIFO	40.2	22.53%	59.4	154,688	6.3	60.0	74.0	4,117	-0.95	2,698
CR	44.4	15.16%	48.9	82,296	3.8	88.6	79.6	3,822	+10.38	3,019
QT	57.5	81.66%	38.4	1,925	0.0	1.4	80.8	2,145	-1.13	3,113
QT, promote < 50 % of window	57.4	89.60%	38.3	524	0.0	1.1	81.0	2,166	+1.17	3,067
QT, promote < 25 % of window	57.2	88.82%	38.4	809	0.0	1.4	80.9	2,200	+0.48	3,045
SLATE, q-time term inert (a)	48.1	24.91%	48.8	91,889	0.0	22.2	79.7	3,843	+10.40	17,776
SLATE, q-time term inert (b)	48.5	24.17%	48.6	91,583	0.0	23.1	79.9	3,765	+8.95	17,626
SLATE, window- relative, untuned fallback (a)	57.4	92.89%	36.9	114	0.0	1.4	80.1	2,154	+0.43	14,739
SLATE, window- relative, untuned fallback (b)	57.7	93.02%	36.9	122	0.0	1.7	80.3	2,106	+0.68	14,781
SLATE, window- relative, tuned fallback	57.3	96.10%	37.2	77	0.0	1.6	80.2	2,184	+1.23	14,617

Table 5. Every configuration on seed 0, resumed from one qt-warmed checkpoint, 180-day window. Tardiness is total lot-days late over the window. Wall clock is for the 180 measured days on 16 cores; solver rows use 8-way family parallelism.

With the window-relative term, two replicates of the solver deliver 57.4 and 57.7 good lots/day (qt: 57.5), 92.89 and 93.02 % on-time (qt: 81.66), cycle time 36.9 days (qt: 38.4), tardiness 114 and 122 lot-days (qt: 1,925), violations at qt’s rate, zero scrap, and stationary WIP. The replicates agree to 0.13 on-time points — far inside the noise floor of §3.4 — so the reversal is real.

6.5 Tuning the baseline, and what that removes

Before treating that as a solver win we made the baseline as strong as we could. The qt in Table 5 promotes any saveable at-risk lot; with a threshold of half the window it reaches 89.60 % on-time and tardiness of 524 (a quarter of the window: 88.82 % and 809). That single threshold **removes the most interesting claim we had made**. We had attributed the solver’s win to rebalancing across a set — lifting the products

running late without hurting those running early, which a ranking supposedly cannot do because it has no way to move slack between products. The thresholded `qt` narrows the per-product spread from 32.9 to 15.8 points; the solver’s untuned-fallback replicates sit at 14.4 and 16.3. The rebalancing was not a property of assignment over a set. It was a property of not promoting lots that were never in danger, and a sort key can be told that in one line.

The margin against the strongest baseline shrinks accordingly: +3.3 on-time points, 4.3× less tardiness, 1.4 days of cycle time.

6.6 The symmetric comparison

That comparison is itself unfair to the solver, in a way that is invisible in the result files. Coverage is 46 %, so **the `qt` fallback decides more than half of every solver run.** The solver rows in §6.4 were produced before the threshold existed: they carried the untuned `qt` on that majority of decisions while being measured against the tuned one. The solver contains the baseline; improving the baseline improves half the solver, and only the baseline had been re-run.

Re-run with both sides tuned, the solver reaches **96.10 % on-time, tardiness 77 lot-days, and a per-product spread of 10.1 points**, at 57.3 good lots/day and stationary WIP. Against the tuned `qt`, that is **+6.5 on-time points, 6.8× less total tardiness, and a spread of 10.1 against 15.8** — so a weaker version of the withdrawn claim returns: the solver does rebalance across products better than a ranking, by about six points of spread rather than by being uniquely capable of it. Both cells are single runs and should be held loosely.

6.7 Per-product results

product	QT	QT tuned	SLATE inert	SLATE fixed (a)	SLATE fixed (b)	SLATE sym
product 9	66.41%	83.77%	11.67%	84.88%	83.27%	89.47%
product 6	66.73%	84.14%	13.64%	88.57%	87.36%	94.19%
product 8	76.69%	86.07%	12.14%	89.67%	90.12%	95.64%
product 5	76.74%	85.89%	13.58%	89.16%	90.87%	93.77%
product 1	76.97%	86.64%	14.10%	92.34%	92.86%	96.41%
product 3	77.38%	86.38%	14.75%	93.15%	92.62%	96.72%
product 2	78.03%	86.74%	15.24%	93.80%	95.29%	96.81%
product 7	98.55%	97.29%	99.43%	99.32%	99.61%	99.61%
product 10	99.32%	99.61%	18.07%	98.74%	99.52%	99.03%
product 4	99.33%	99.42%	19.62%	99.32%	98.75%	99.32%
spread (max – min)	32.9 pts	15.8 pts	87.8 pts	14.4 pts	16.3 pts	10.1 pts
fab-wide	81.66%	89.60%	24.91%	92.89%	93.02%	96.10%

Table 6. On-time delivery per product, seed 0, every configuration, products ordered by their result under `qt`. “SLATE fixed” is the window-relative term with the untuned fallback (two replicates); “SLATE sym” is the same with the tuned fallback.

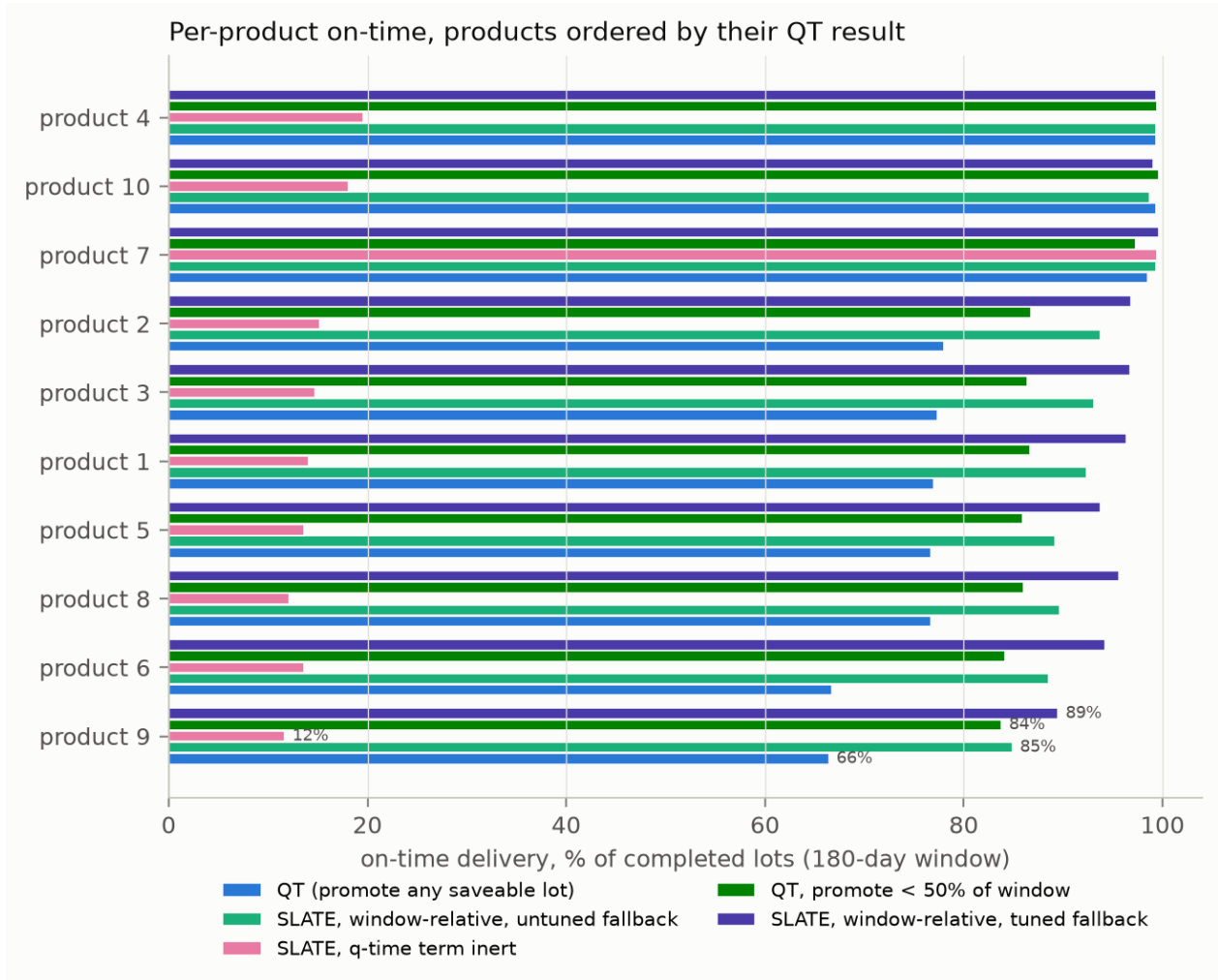


Figure 4. Per-product on-time delivery for the principal configurations. The solver with its inert term (pink) flattens every product but one to 12–20 %. The tuned *qt* (green) and the corrected solver (aqua, violet) both lift the laggards without sacrificing the leaders; the symmetric solver run lifts them furthest.

Two structural facts about the products explain why this is a due-date allocation problem. First, the lateness is marginal: under *qt*, late lots miss by 0.47 to 1.40 days on cycle times of 23–55 days — a 1–5 % overshoot. Second, it is not congestion: products 9 and 10 have identical cycle times (38.4 days) and are 33 points apart on on-time; product 4 is slower than product 6 and 33 points more punctual. The laggards are promised sooner relative to their routes, and the leaders have days of margin. Moving a fraction of a day of service from one to the other is the intended shape of the assignment formulation — and, as §6.5 shows, most of it is also reachable by a threshold.

product	route steps	q-time steps	QT CT	QT tard.	QT-tuned CT	QT-tuned tard.	SLATE-sym CT	SLATE-sym tard.
product 9	384	23	38.4	162	38.2	47	37.6	4
product 6	293	27	29.4	204	29.2	47	27.7	4
product 8	375	20	36.8	305	36.5	83	35.9	8
product 5	242	11	22.8	287	22.6	84	22.6	15
product 1	521	29	48.5	283	48.3	77	47.4	12
product 3	583	41	54.9	306	54.6	90	53.0	8
product 2	529	38	51.1	319	50.8	81	48.8	11
product 7	353	24	33.0	5	33.4	6	32.0	3
product 10	390	26	38.4	33	38.5	2	37.5	9
product 4	343	25	30.5	23	30.7	8	29.0	2

Table 7. Per-product cycle time (days) and total tardiness (lot-days) for the untuned rule, the tuned rule, and the symmetric solver run, with route length and queue-time step count for context.

6.8 Total lateness

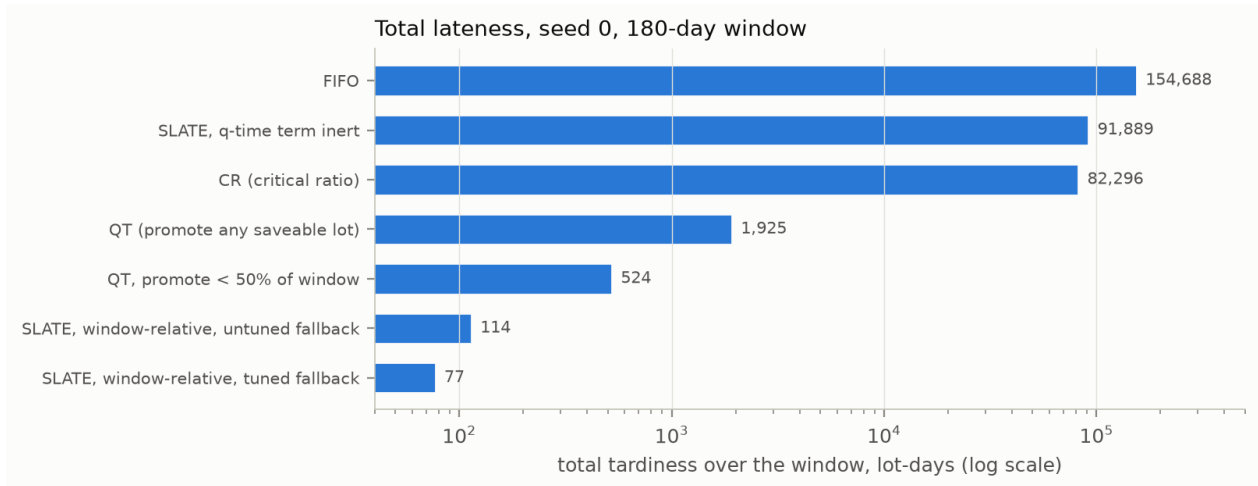


Figure 5. Total tardiness over the window, seed 0, log scale. Three orders of magnitude separate the diverging rules from the best configurations.

Tardiness is the metric on which the solver’s advantage stayed wide after every correction to the baseline. It measures not only how many dates are missed but by how much, and the corrected solver misses fewer and misses them by far less. We do not have a mechanism for this beyond the observation that the objective’s due-date curve is steep below $CR = 1$, so a lot that is going to be late is pushed hard; it is the most interesting open question the results leave.

6.9 Cost

A `qt` run of the 180-day window takes $\approx 3,100$ s on this machine. The corrected solver takes $\approx 14,700$ s with 8-way family parallelism, an average of about three cores busy — **$\approx 4.7\times$ the wall clock and on the order of $15\times$ the CPU time** — for a decision that is

46 % the solver’s and 54 % its fallback’s. Profiling attributes 60 % of a solver run to CP-SAT itself and 13 % to marshalling state across the Python/C++ boundary; the belief that the boundary was the bottleneck was true of an earlier version and had outlived the fix that made it false.

7. Threats to validity

7.1 Sample size

The rule result rests on three seeds and is the strongest thing here. The solver results rest on **one seed**, with two replicates for the inert and corrected configurations and one run for the symmetric configuration. Each tuned-qt variant is one run. Given that identical solver runs vary by 7 % in lots on a short probe (§3.4), the 3.3-point and 6.5-point margins should be read as estimates with an error bar of a point or two, not as measurements to the decimal.

7.2 One fab, one scenario

Everything is LVHM. Tools within a family are identical; reticles never bind; transport is free. That is precisely the environment in which an assignment solver has least to offer, and nothing here speaks to the high-volume/low-mix scenario or to a fab with a binding coupling constraint.

7.3 The short-window trap

window read	SLATE-inert thr/day	SLATE-inert on-time %	QT thr/day	QT on-time %
days 90–110	56.4	82.1	58.5	83.3
days 90–150	50.2	39.4	56.1	91.4
days 90–210	48.6	27.9	57.1	77.1
days 90–270	48.1	23.7	57.6	82.3

Table 8. One solver run (inert term) read over growing windows, against qt over the same days. Values are means of the simulator’s trailing-day samples.

One SLATE run read over growing windows (means of trailing-day samples)

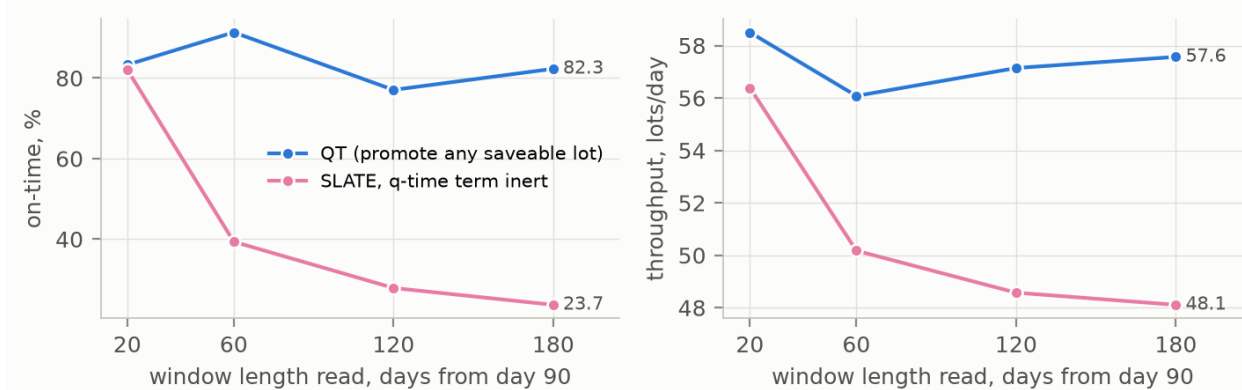


Figure 6. The same solver run read over windows of 20, 60, 120 and 180 days. At 20 days it is within two lots/day and 1.3 on-time points of QT — a result anyone would report as "no material difference." The gap then widens monotonically as the backlog compounds.

At 20 days the failing solver configuration is indistinguishable from `qt`. The decay is monotonic, so it is not noise; it is a backlog building. We nearly published the wrong answer twice on this account: a short probe of the corrected solver read as an outright win before the long run showed a smaller one, and a 20-day probe of the inert solver read as level. The general rule is that a window long enough to be convenient is not long enough to be right, and the dangerous case is not the short window that points the wrong way but the one that looks acceptable while the mechanism that ruins the run is still building.

7.4 Baseline containment

The solver is a solver plus a fallback, so it contains the baseline. Any comparison must tune both or neither; tuning one silently measures the handicap, and nothing in a result file records which version of the rule a solver row carried. We fell into this once (§6.6). It generalises to any hybrid policy with a rule underneath.

7.5 Unaudited coefficients

One constant, wrong by two orders of magnitude relative to the data, inverted a replicated, written-up conclusion. The objective has several more — the critical-ratio curve's breakpoints and 50× cap, the ageing rate, the downstream-congestion weight, the two disagreeing queue-time constants — and none has been checked the same way. Until they are, "the solver lost" on the earlier constraint classes is a weaker statement than it reads, and the margin reported here is a property of this objective rather than of assignment as such.

8. What we learned about measurement

Every wrong answer this work produced came from one of two places: a metric that could not respond to the thing being changed, or a number nobody had checked. In order: reading a drain of WIP as throughput; averaging a per-product effect across the fab; measuring on-time while the fab was diverging; comparing a two-day utilisation against a ninety-day one; mixing counters accumulated from day 0 with a window-

scoped throughput; sizing the solver’s opportunity from one seed immediately after writing “one seed” down as the caveat; reaching for a bit-identical fingerprint as a validation standard on a solver whose budget is wall-clock; and a queue-time term calibrated in minutes for windows measured in days.

Every fix was a control or an invariant — a detection-only arm digit-identical to no enforcement, a conservation identity, an admissibility gate read before any effect, a noise floor measured before a margin was claimed — never a cleverer metric. We record this not as confession but because the pattern transfers: in a system this coupled, the first thing to check about any surprising number is whether it could have moved at all.

9. Next steps

1. **Replicate the solver on seeds 1 and 2**, and the symmetric configuration at seed 0, so that every cell in Table 5 has an error bar.
2. **Audit the remaining objective coefficients** by the same method that found this one: compute what value a typical lot actually receives and compare it to the scale of the competing terms.
3. **Fit the objective rather than guess it**. Treat its coefficients as a vector to be optimised against full-window runs — each evaluation is a real run, because §7.3 shows a short one cannot be trusted. An imitation floor first: if the objective cannot be fitted to reproduce `qt`, it is misspecified, which is a one-run diagnostic.
4. **Give CP-SAT a deterministic budget**, so runs replay and validation can be exact.
5. **The high-volume scenario**, where tools within a family are not interchangeable and the coupling the assignment formulation exists to exploit is present.
6. **Re-test the earlier negatives** — qualification and reticles — with the corrected objective, before letting them stand as evidence.

10. Reproducibility

All code, data, result files and decision records are at <https://github.com/altonalexander/fab-optimization>, Apache-2.0. Release **v0.2.0** (merge commit `09065d2`) contains every experiment through §6.5; the symmetric comparison of §6.6 and the run-store importer are in the commits following it on `main`. This document is built from commit `ecf1b9f` by `docs/paper/build/: paper_data.py` consolidates the result files, `figures.py` renders every figure, and `build_pdf.py` generates every table and this PDF, so no number in the tables was typed.

The simulator is PySCFabSim at its pinned upstream commit with the divergences listed in `baselines/pyscfabsim/UPSTREAM.md`. The solver uses OR-Tools **9.15.6755** with CP-SAT, 1 search worker, a 5 ms per-family budget, relative gap 0.02, and a 60-second simulated planning cycle. A typical row is produced by

```
bench/tools/compare.py --days 270 --warmup-days 90 --warmup-dispatcher qt \  
  --rules slate --slate-fallback qt --cqt --cqt-scale 10 \  
  --starts-scale 1.00 --threads -8 --out RESULT.json
```

with `QT_PROMOTE_FRAC=0.50` in the environment for the tuned rule. Result files for every row in this paper are under `bench/results/cliff/`. The decision records `docs/adr/0016`

(queue-time enforcement, the `qt` rule) and `docs/adr/0017` (the operating-point search, the falsified verdict, its correction) preserve each conclusion as it was written, including the ones later shown to be wrong; `docs/adr/0009` records the profile and the parallel planner, and `docs/adr/0008` the simulator’s simplifications. A plain-language account of the work is in `docs/notes/`.

Acknowledgements

The simulator is the work of the Research Group Production Systems; the testbed of Kopp, Hassoun, Kalir and Mönch; the solver of the OR-Tools team. The experiments, code and analysis in this paper were carried out with an AI coding assistant (Claude, Anthropic) operating the simulator and writing the analysis tooling under the author’s direction; the decisions, the questions that turned the work — the queue-time-aware rule, the fallback asymmetry, the standard of equivalence — and the responsibility for the claims are the author’s.

References

- [1] S. S. Panwalkar and W. Iskander, “A survey of scheduling rules,” *Operations Research*, vol. 25, no. 1, pp. 45–61, 1977.
- [2] J. H. Blackstone, D. T. Phillips, and G. L. Hogg, “A state-of-the-art survey of dispatching rules for manufacturing job shop operations,” *International Journal of Production Research*, vol. 20, no. 1, pp. 27–45, 1982.
- [3] R. Uzsoy, C.-Y. Lee, and L. A. Martin-Vega, “A review of production planning and scheduling models in the semiconductor industry, part I: system characteristics, performance evaluation and production planning,” *IIE Transactions*, vol. 24, no. 4, pp. 47–60, 1992.
- [4] R. Uzsoy, C.-Y. Lee, and L. A. Martin-Vega, “A review of production planning and scheduling models in the semiconductor industry, part II: shop-floor control,” *IIE Transactions*, vol. 26, no. 5, pp. 44–55, 1994.
- [5] L. Perron and V. Furnon, OR-Tools, version 9.15.6755, Google. <https://developers.google.com/optimization/>
- [6] A. Alexander, fab-optimization: predictive lot dispatching for a 300 mm semiconductor fab, 2026. <https://github.com/altonalexander/fab-optimization>
- [7] D. Kopp, M. Hassoun, A. Kalir, and L. Mönch, “SMT2020 — A semiconductor manufacturing testbed,” *IEEE Transactions on Semiconductor Manufacturing*, 2020. doi:10.1109/TSM.2020.3001933
- [8] Research Group Production Systems, PySCFabSim, MIT licence. <https://github.com/prosysscience/PySCFabSim-release>
- [9] What PySCFabSim simplifies, and what that hides, decision record 0008 in [6].